

INFORME TRABAJO FINAL UNQ SHOP

Trabajo final: Programación con Objetos 2 - Primer Cuatrimestre 2026



**Sandoval Thomas, Sarracini Martin, Lopetegui
Juan Manuel**

2 de julio de 2026

Información de contacto

thomassandoval11@gmail.com

martinsarraciniontivero@gmail.com

lopeteguijuanma@gmail.com

INTRODUCCIÓN

Este documento tiene como objetivo registrar las decisiones de diseño tomadas por el grupo, sobre cómo modelar el trabajo final de la materia Programación con Objetos 2, para cada módulo del trabajo se indica el patrón aplicado, los roles según el Gamma et al. (GoF) y las decisiones de diseño tomadas con su justificación.

RESUMEN DE PATRÓN POR MÓDULO

Módulo	Patrón	Rol principal
2.1 Catálogo de productos	Composite	Componer productos y paquetes uniformemente
2.2 Ciclo de vida del pedido	State	Variar el comportamiento según el estado
2.3 Métodos de envío	Strategy	Intercambiar algoritmos de cálculo de costo
2.4 Métodos de pago	Template Method	Fijar el orden del proceso, variar los pasos
2.5 Notificaciones del pedido	Observer	Desacoplar el pedido de los subsistemas que reaccionan
2.6 Búsqueda en el catálogo	Specification	Componer criterios de búsqueda combinables
2.7 Stock (Sucursal / DepositoGeneral)	—	Inventario por ubicación, separado del catálogo

2.8 Reportes	Visitor	Separar el formato de salida de la estructura de datos
2.9 Flujo de ventas (LineaDePedido)	—	Congelar precio y fecha para el reporte histórico

2.1 CATÁLOGO DE PRODUCTOS

Problema que resuelve: el catálogo contiene productos individuales y paquetes, y un paquete puede contener otros paquetes anidados. El cliente que navega el catálogo no debe distinguir si consulta un producto simple o un paquete: ambos responden al mismo contrato.

Roles GoF:

- Component: ItemCatalogo (clase abstracta)
- Leaf: Producto
- Composite: Paquete

Decisiones:

1. ItemCatalogo es una clase abstracta, no una interfaz. Se eligió clase abstracta para compartir el estado común (nombre, descripcion, unidadesVendidas, precioPromedio) y la implementación concreta de getNombre() y getDescripcion(), evitando duplicación entre Producto y Paquete. El costo asumido es la herencia simple de Java: ni Producto ni Paquete pueden extender otra clase. Se aceptó ese costo porque ninguna entidad del catálogo necesita pertenecer a otra jerarquía.
2. getPrecioBase() y aceptar() quedan abstractos. El precio se calcula distinto en cada subclase: Producto devuelve su precioFinal, mientras que Paquete lo calcula recursivamente como la suma de los precios de sus ítems multiplicada por (1 – descuento). El método aceptar() queda abstracto por el double dispatch del patrón Visitor (cada subclase invoca visitar(this) con su tipo concreto).
3. El cálculo de precio del paquete es recursivo y uniforme. Como cada ítem responde getPrecioBase() polimórficamente, el cálculo funciona automáticamente con paquetes anidados sin código especial para tratar un paquete-dentro-de-paquete.
4. Atributos dinámicos mediante Map<String, Object>. El enunciado exige poder

agregar atributos dinámicos (alto, ancho, peso) que no sean campos de una clase del modelo. Se modelaron con un `Map<String, Object>` en `Producto`, lo que permite extender los atributos sin modificar la clase. Se descartó usar constructores con parámetros fijos porque no escalan: obligarían a predefinir todos los atributos posibles en la firma.

5. Validación por asignación de valor. El método `validar()` solo comprueba que los atributos obligatorios tengan un valor asignado, tanto los fijos (nombre, sku) como los dinámicos obligatorios (una lista de claves requeridas que se chequean contra el mapa). Según la aclaración del enunciado, no se valida formato, rango ni tipo, únicamente la asignación.
6. Catalogo mantiene los ítems por agregación. La relación `Catalogo → ItemCatalogo` (campo `items: List<ItemCatalogo>`) es una agregación: los ítems existen independientemente del catálogo.

2.2 CICLO DE VIDA DEL PEDIDO - STATE

Problema que resuelve: un pedido modifica su comportamiento a lo largo de su ciclo de vida. Cada estado habilita operaciones distintas, y una operación inválida para el estado vigente debe rechazarse.

Roles GoF:

- Context: Pedido
- State: `EstadoPedido` (interfaz), con una clase abstracta intermedia `EstadoBase`
- ConcreteState: Borrador, Confirmando, EnPreparacion, Enviado, Entregado, Cancelado

Decisiones de diseño:

1. Interfaz `EstadoPedido` + clase abstracta `EstadoBase`. Se introdujo una clase abstracta `EstadoBase` (que implementa `EstadoPedido`) entre la interfaz y los estados concretos. `EstadoBase` implementa todas las operaciones lanzando `OperacionInvalidaException` por defecto, y declara abstracto el método `nombreEstado()`. Cada estado concreto extiende `EstadoBase` y solo sobrescribe las operaciones que son válidas en ese estado. La alternativa (interfaz pura) obligaba a que cada estado escribiera las siete operaciones aunque solo usara una o dos; con `EstadoBase` los estados terminales (Entregado, Cancelado) quedan triviales, declarando únicamente su nombre. La jerarquía resultante tiene tres niveles: interfaz → abstracta → concretos.

2. Las transiciones viven en cada clase de estado. Cada estado concreto encapsula su propio comportamiento y sabe a qué estados puede transicionar, en lugar de centralizar las transiciones en el Pedido. Esto es lo más fiel al patrón y mantiene cada regla de transición junto al estado que la posee.
3. Los estados son stateless (sin atributos propios). Toda la data del pedido (ítems, observers, estado actual) vive en el contexto Pedido. Los estados reciben el pedido como parámetro para operar sobre él. Esto permite que los estados no necesiten guardar información y mantiene una única fuente de verdad.
4. La lógica diferencial de cancelación vive en el estado origen. Cada estado resuelve su propia cancelación antes de transicionar a Cancelado, de modo que Cancelado no necesita saber de dónde vino. Según la versión vigente del enunciado: cancelar desde EnPreparacion repone el stock y reembolsa el total (producto + envío); cancelar desde Enviado reembolsa solo el producto, no el envío, y repone stock. El decremento de stock ocurre al confirmar, en Borrador.confirmar().
5. Operaciones inválidas mediante excepción de dominio. Una operación no permitida para el estado vigente lanza OperacionInvalidaException (extiende RuntimeException, no chequeada). El mensaje incluye el nombre del estado para facilitar el diagnóstico.
6. Pedido → EstadoPedido es una asociación. El campo estado se modela como asociación (no dependencia) porque el pedido mantiene una referencia al estado actual.

2.3 MÉTODOS DE ENVÍO - STRATEGY

Problema que resuelve: el costo y las condiciones de envío varían según el método elegido, y cada método usa criterios de cálculo completamente distintos.

Roles GoF:

- Strategy: MetodoDeEnvio (interfaz)
- ConcreteStrategy: EnvioEstandar, EnvioExpressImpl, RetiroEnSucursal
- Context: el Pedido (tiene un MetodoDeEnvio y lo usa en getCostoEnvio())

Decisiones de diseño:

1. Contrato uniforme pese a firmas externas distintas. La interfaz MetodoDeEnvio expone calcularCosto(Pedido) y estimarDiasDeEntrega(Pedido). Cada estrategia extrae del pedido lo que necesita: el estándar el peso, el express el valor monetario, el retiro consulta la sucursal.

2. Las librerías externas se modelan como interfaces y se inyectan. `CorreoArgentina` (usada por `EnvioEstandar`) y `EnvioExpress` (usada por `EnvioExpressImpl`) son interfaces de servicios externos que el dominio no implementa —el enunciado dice "sólo debe utilizar"—. Se reciben por constructor, lo que las hace mockeables en los tests.
3. `EnvioEstandar` calcula por peso: `correo.estimarEnvio(pedido.getPesoTotal(), direccion)`. Estima 7 días.
4. `EnvioExpressImpl` calcula por valor monetario: `apiExpress.calcularCosto(pedido.getSubtotalProductos())`. Usa el subtotal de productos (no el total) para evitar la recursión con `getTotal()`, que a su vez llama al costo de envío. Garantiza 1 día.
5. `RetiroEnSucursal` siempre devuelve costo cero y estima los días con `sucursal.estimarDiasDeRetiro()` (el máximo entre todos los ítems): 0 si hay stock local, 3 si requiere traslado desde otra sucursal. Guarda la Sucursal elegida, que Ecommerce consulta para descontar el stock del lugar correcto.

2.4 MÉTODOS DE PAGO - TEMPLATE METHOD

Problema que resuelve: el procesamiento de un pago sigue siempre los mismos pasos en el mismo orden (validar → reservar → ejecutar → notificar), pero cada medio de pago implementa esos pasos de manera diferente.

Roles GoF:

- AbstractClass: `MetodoPago`
- ConcreteClass: `TarjetaCredito`, `TransferenciaBancaria`, `BilleteraVirtual`

Decisiones de diseño:

1. `MetodoPago` es una clase abstracta. El patrón Template Method requiere una clase abstracta que contenga el método plantilla. La interfaz no sirve porque el método plantilla debe tener implementación.
2. El método plantilla `procesarPago()` es fijo y no se sobrescribe. Define el orden invariante de los pasos: validar → reservar → ejecutar → notificar. Las subclasses nunca lo tocan.

3. Tres pasos abstractos obligatorios. `validarDatos()`, `reservarFondos()` y `ejecutarTransaccion()` son abstractos (primitive operations). Las tres subclases están obligadas a implementarlos. En `TransferenciaBancaria`, `reservarFondos()` se implementa vacío porque el enunciado indica que no aplica, pero igual debe definirse porque la clase abstracta lo exige.
4. `notificarResultado()` es un hook con implementación por defecto. Es el único paso con sobreescritura opcional. La implementación por defecto registra el código de transacción; las subclases la personalizan solo si lo necesitan: `TarjetaCredito` emite un cupón imprimible, `TransferenciaBancaria` genera un comprobante CBU, `BilleteraVirtual` envía una notificación push y aplica cashback. Una subclase futura que no necesite personalizar la notificación usaría el default sin sobreescribirlo.
5. Visibilidad `protected` en los pasos. Los pasos son parte del contrato interno entre la clase abstracta y sus subclases; no se exponen públicamente.

2.5 NOTIFICACIONES DEL PEDIDO - OBSERVER

Problema que resuelve: cuando un pedido cambia de estado, distintos subsistemas deben reaccionar de forma independiente. El módulo de pedidos no debe conocer ni depender de esos subsistemas, y deben poder agregarse nuevos a futuro.

Roles GoF:

- Subject: Pedido
- Observer: `ObserverPedido` (interfaz)
- ConcreteObserver: `NotificadorMail`, `GeneradorFactura`, `Fidelizacion`

Decisiones de diseño:

1. Pedido es el sujeto, mantiene la `List<ObserverPedido>` (`observersNotis`) y expone `agregarObserver()`, `quitarObserver()` y `notificarObservers()`.
2. La notificación se dispara en cada transición de estado. Cada estado, al transicionar, llama a `pedido.notificarObservers(estadoViejo, estadoNuevo)`. El método recorre los observers y les pasa ambos estados.
3. Extensibilidad. Agregar un subsistema es crear una clase que implemente `ObserverPedido` y registrarla, sin modificar `Pedido`.

2.6 BÚSQUEDA EN EL CATÁLOGO - SPECIFICATION

Problema que resuelve: el sistema debe permitir buscar ítems aplicando criterios simples o combinándolos con operadores lógicos (AND, OR, NOT), con anidamiento libre de cualquier profundidad. El módulo de búsqueda no debe decidir cómo evaluar los ítems: esa responsabilidad recae en el criterio recibido.

Roles GoF (variante de Composite aplicada a predicados):

- Component: CriterioBusqueda (interfaz)
- Leaf: CriterioNombre, CriterioPrecioMaximo, CriterioCategoria, CriterioDisponibilidad
- Composite: CriterioAnd, CriterioOr, CriterioNot

Decisiones de diseño:

1. Interfaz única CriterioBusqueda con un método `satisface(ItemCatalogo)`. Tanto los criterios simples como los compuestos la implementan. Esa uniformidad es lo que habilita el anidamiento recursivo.
2. Los criterios simples encapsulan su dato de comparación. CriterioNombre guarda el texto (normalizado a minúsculas para comparación sin distinción de mayúsculas), CriterioPrecioMaximo el precio máximo, CriterioCategoria la categoría. CriterioDisponibilidad recibe el depósito general y consulta si tiene stock.
3. Los criterios compuestos también son criterios. CriterioAnd y CriterioOr reciben dos criterios y los combinan con `&&` / `||`; CriterioNot envuelve exactamente un criterio y lo niega. Esto permite el anidamiento de criterios.
4. CriterioCategoria y los paquetes. Como la categoría es un atributo propio de Producto y no de ItemCatalogo, CriterioCategoria verifica con `instanceof Producto` y, por decisión de diseño, un Paquete nunca satisface el criterio de categoría (no tiene categoría propia). El enunciado no define la categoría de un paquete, por lo que se optó por esta interpretación.
5. La búsqueda vive en una clase `BuscadorCatalogo` separada. Recibe la lista de

ítems y un único `CriterioBusqueda`, y filtra con streams (`filter(criterio::satisface)`). El buscador no toma ninguna decisión sobre cómo evaluar: delega en el criterio recibido. Si el cliente quiere combinar varios criterios, construye el criterio compuesto antes de invocar la búsqueda.

6. `Ecommerce` como raíz del sistema. Es la clase que contiene el catálogo maestro (`List<ItemCatalogo>`), la red de sucursales (`List<Sucursal>`) y el `BuscadorCatalogo`. Expone `buscarEnCatalogo(criterio)` delegando en el buscador. Actúa como punto de entrada y fachada del sistema.

2.8 REPORTES - VISITOR

Problema que resuelve: el reporte de productos más vendidos puede exportarse en varios formatos (texto plano, CSV, HTML). El núcleo del reporte (qué datos recolectar y cómo estructurarlos) debe ser independiente del formato de salida.

Roles GoF:

- Visitor: `ReporteVisitor` (interfaz)
- ConcreteVisitor: `ReporteTXT`, `ReporteCSV`, `ReporteHTML`
- Element: `ItemCatalogo`
- ConcreteElement: `Producto`, `Paquete`

Decisiones de diseño:

1. El visitor representa el formato de exportación. `ReporteVisitor` declara `visitar(Producto)`, `visitar(Paquete)` y `obtenerResultado()`. Cada visitor concreto sabe cómo representar cada tipo de ítem en su formato.
2. Double dispatch mediante `aceptar()`. `ItemCatalogo` agrega `aceptar(ReporteVisitor)` a su contrato. `Producto` y `Paquete` lo implementan llamando `v.visitar(this)`, lo que resuelve en tiempo de ejecución qué método del visitor se invoca según el tipo concreto del ítem.
3. Los visitors acumulan el resultado. Cada visitor concreto (`ReporteTXT`, `ReporteCSV`, `ReporteHTML`) construye la salida en un `StringBuilder` y la devuelve con `obtenerResultado()`. El CSV abre con una fila de encabezado; el HTML abre la tabla en el constructor y la cierra en `obtenerResultado()`. Los importes se formatean con `Locale.US` para usar punto decimal (necesario sobre todo en CSV,

donde la coma es separador de campos).

4. `GeneradorReporteVentas` orquesta el reporte. Recibe por parámetro los pedidos, los ítems, el rango de fechas (desde/hasta) y el visitor. Su flujo: (a) reinicia las ventas acumuladas de todos los ítems; (b) filtra los pedidos entregados dentro del período y registra sus ventas en cada ítem; (c) descarta los ítems sin ventas y ordena el resto por unidades vendidas de mayor a menor; (d) invoca `item.aceptar(visitor)` sobre cada uno (double dispatch); (e) devuelve `visitor.obtenerResultado()`.
5. Datos de venta en el catálogo. Para que el Visitor visite `Producto/Paquete` directamente (manteniendo el patrón limpio), `ItemCatalogo` acumula los datos de venta del período: `registrarVenta(cantidad, precioUnitario)` acumula unidades y suma de precios; `getPrecioPromedio()` devuelve el promedio ponderado; `reiniciarVentas()` limpia los acumuladores. Como son acumuladores globales, el generador los reinicia antes de cada reporte para que reflejen solo el período consultado. Se asumió este costo (estado temporal en el catálogo) a cambio de mantener el Visitor visitando los ítems directamente, en lugar de introducir una estructura intermedia.
6. El "precio promedio efectivamente cobrado" puede diferir del precio base por descuentos de paquete. Se calcula como promedio ponderado de los precios congelados al momento de la venta (sección 2.9).
7. Extensibilidad de formatos. Agregar un nuevo formato de salida es crear una nueva clase que implemente `ReporteVisitor`, sin modificar `Producto`, `Paquete` ni `ItemCatalogo`.

2.9 FLUJO DE VENTAS EN EL PEDIDO

Problema que resuelve: el reporte de productos más vendidos necesita saber qué se vendió, en qué cantidad, a qué precio y cuándo. Esos datos los aporta el pedido. El "precio efectivamente cobrado" debe ser el del momento de la venta, no el precio actual del ítem (que pudo cambiar).

Decisiones de diseño:

1. `LineaDePedido` como unidad de venta. El pedido guarda una `List<LineaDePedido>`, donde cada línea tiene el ítem, la cantidad y el precio unitario cobrado. Reemplaza a la antigua `List<ItemCatalogo>`. Agregar el mismo ítem dos veces incrementa la cantidad de su línea en lugar de duplicarla.
2. El precio se congela al confirmar. `LineaDePedido.capturarPrecio()` guarda el

precioFinal() del ítem en el momento de confirmar el pedido (Borrador.confirmar() invoca pedido.capturarPrecios()). Así un cambio posterior de precio en el catálogo no altera lo ya vendido. Se eligió el precio unitario (no el total de la línea) para facilitar el cálculo del promedio en el reporte.

3. La fecha de entrega se registra al entregar. Al pasar a Entregado (Enviado.entregar() invoca pedido.registrarEntrega()), el pedido guarda su fechaEntrega. Un pedido no entregado tiene fecha nula.
4. Solo los pedidos entregados cuentan como venta. El reporte filtra por estado Entregado y por fecha dentro del período. Esto resuelve el caso de cancelación: un pedido cancelado captura su precio al confirmar pero nunca llega a Entregado, por lo que el reporte lo ignora automáticamente, sin necesidad de "deshacer" ventas.
5. Dos precios distintos según el propósito. El getSubtotal() de LineaDePedido usa el precio congelado (precioCobrado) y sirve al reporte histórico. En cambio Pedido.getSubtotalProductos() calcula sobre el precio actual del ítem (item.precioFinal()) y sirve para el total a cobrar (getTotal()), que necesita el precio vigente al momento de facturar, no uno congelado que solo existe tras confirmar. getTotal() suma ese subtotal más el costo de envío.
6. El pago se desacopla del estado (decisión de diseño). Borrador.confirmar() transiciona y captura precios, pero NO dispara el pago. El cobro (pedido.procesarPago()) se orquesta desde Ecommerce.confirmarPedido(), junto con el descuento de stock. Esto evita acoplar el módulo State al módulo de pago: el estado solo se ocupa de la transición, y Ecommerce coordina el flujo completo (confirmar + pagar + descontar stock). Si falta el método de pago, procesarPago() lanza PagoNoSeleccionadoException (excepción de dominio).

DECISIONES TRANSVERSALES

- Pedido como entidad central. Concentra el estado (State), las líneas de venta (LineaDePedido), los observers (Observer), y ahora el método de envío y de pago seleccionados. Es el nodo más conectado del modelo, coherente con su rol de orquestador del flujo de compra.
- Envío y pago acoplados al pedido por selección, no por lógica. Pedido guarda un MetodoDeEnvio y un MetodoPago (los medios elegidos), pero la lógica vive en cada estrategia/subclase. MetodoDeEnvio.calcularCosto(Pedido) recibe el pedido

por parámetro, generando la dependencia de envío hacia pedido. El cobro se dispara desde Ecommerce, no desde el estado (ver 2.9).

- Las notificaciones del módulo 2.5 se implementaron con un `system.printout`, para facilitar el diseño.